

Unit 14: Network File Systems

CONTENTS

Objectives

Introduction

14.1 Setting Up an NFS Client

14.2 Setting Up an NFS Server

14.3 Testing the Server Setup

14.4 Automount: Automatically Mounts Directory Hierarchies

Summary:

Keywords:

Self Assessment

Answers for Self Assessment

Review Questions:

Further Readings

Objectives

After studying this unit, you will be able to:

- Understand NFS
- Plan NFS installation
- Configure NFS server and client
- Use automount service
- Examine NFS security

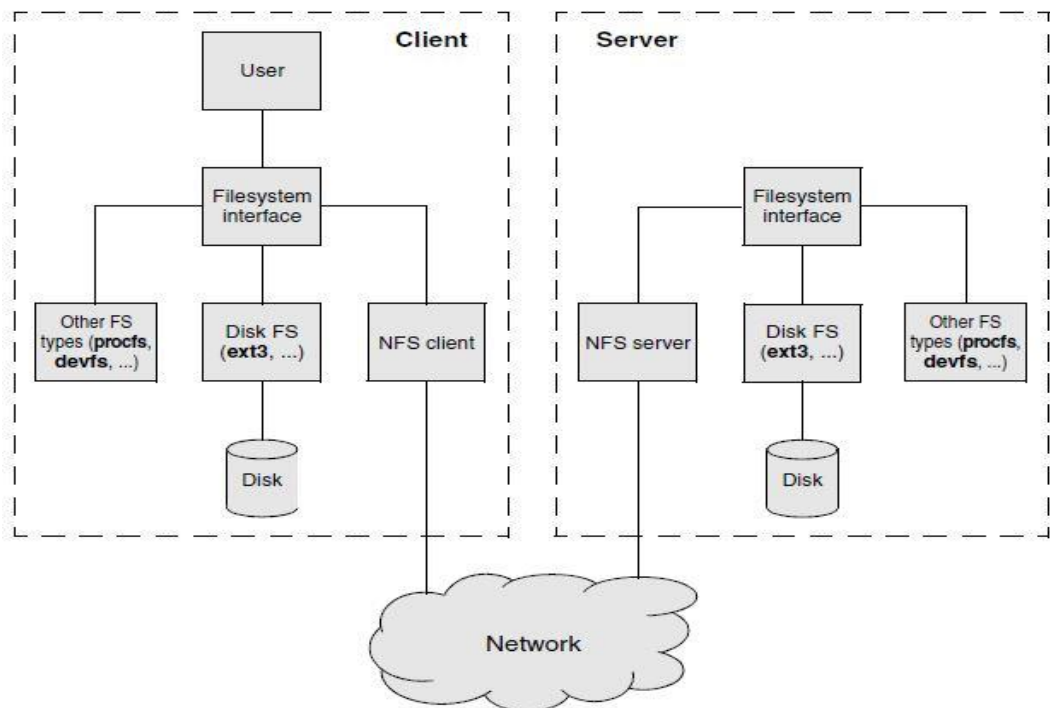
Introduction

NFS stands for Network Filesystem protocol. It is a UNIX de facto standard originally developed by Sun Microsystems. It allows a server to share selected local directory hierarchies with client systems on a heterogeneous network. NFS runs on UNIX, DOS, Windows, VMS, Linux, and more. Files on the remote computer (the fileserver) appear as if they are present on the local system (the client). The physical location of a file is irrelevant to an NFS user.

NFS reduces storage needs and system administration workload. As an example, each system in a company traditionally holds its own copy of an application program. To upgrade the program, the administrator needs to upgrade it on each system. NFS allows you to store a copy of a program on a single system and give other users access to it over the network. This scenario minimizes storage requirements by reducing the number of locations that need to maintain the same data. In addition to boosting efficiency, NFS gives users on the network access to the same data (not just application programs), thereby improving data consistency and reliability. By consolidating data, NFS reduces administrative overhead and provides a convenience to users. The flow of data from a client to server can be seen in the figure given below.

An NFS directory hierarchy appears to users and application programs as just another directory hierarchy. By looking at it, you cannot tell that a given directory holds a remotely mounted NFS directory hierarchy and not a local ext3 filesystem. The NFS server translates commands from the client into operations on the server's filesystem.

- 1) **Diskless systems:** In many computer facilities, user files are stored on a central fileserver equipped with many large-capacity disk drives and devices that quickly and easily make backup copies of the data. A diskless system boots from a fileserver (netboots), a CD, or a floppy diskette and loads system software from a fileserver. The Linux Terminal Server Project (ltsp.org) Web site says it all: "Linux makes a great platform for deploying diskless workstations that boot from a network server. The LTSP is all about running thin client computers in a Linux environment. "Because a diskless workstation does not require a lot of computing power, you can give older, retired computers a second life by using them as diskless systems.



- 2) **Netboot/PXE:** You can netboot systems that are appropriately set up. Fedora/RHEL includes the PXE (Preboot Execution Environment) server package for netbooting Intel systems. Older systems sometimes use tftp (Trivial File Transfer Protocol) for netbooting. Non-Intel architectures have historically included netboot capabilities, which Fedora/RHEL also supports. You can build the Linux kernel so that it mounts root (/) using NFS. Given the many ways to set up a system, the one you choose depends on what you want to do. See the Remote-Boot mini-HOWTO for more information.
- 3) **Dataless systems:** Another type of Linux system is a dataless system, in which the client has a disk but stores no user data (only Linux and the applications are kept on the disk). Setting up this type of system is a matter of choosing which directory hierarchies are mounted remotely.

df: shows where directory hierarchies are mounted: The df utility displays a list of the directory hierarchies available on the system, along with the amount of disk space, free and used, on each. The -h (human) option makes the output more intelligible. Directory hierarchy names that are prepended with hostname: are available through NFS.

- 4) **Errors:** Sometimes you may lose access to remote files. For example, a network problem or a remote system crash may make these files temporarily unavailable. When you try to access a remote file in these circumstances, you get an error message, such as NFS server speedy not responding. When the local system can contact the remote server again, you see another message, such as NFS server speedy OK. Setting up a stable network and server (or not using NFS) is the best defense against these kinds of problems.
- 5) **Security:** NFS is based on the trusted-host paradigm and therefore has all the security shortcomings that plague other services based on this paradigm. In addition, NFS is not encrypted. Because of these issues, you should implement NFS on a single LAN segment only, where you can be (reasonably) sure that systems on a LAN segment are what they claim to be. Make sure a firewall blocks NFS traffic from outside the LAN and never use NFS over the Internet.

14.1 Setting Up an NFS Client

The Prerequisites for setting up an NFS client is to install the following packages:

nfs-utils

system-config-nfs (optional)

Under RHEL, the portmap utility must be running to enable reliable file locking. Under FEDORA, this function is served by rpcbind.

JumpStart I: Mounting a Remote Directory Hierarchy

To set up an NFS client, mount the remote directory hierarchy the same way you mount a local directory hierarchy.

- **mount: Mounts a Remote Directory Hierarchy**

There are various assumption for mounting a remote directory hierarchy. These are: (1) Speedy is on the same network as the local system and is sharing /home and /export with the local system.

(2) The /export directory on speedy holds two directory hierarchies that you want to mount: /export/progs and /export/oracle.

(3) The example mounts speedy's /home directory on /speedy.home on the local system, /export/progs on /apps, and /export/oracle on /oracle.

First use mkdir to create the directories that are the mount points for the remote directory hierarchies:

- **# mkdir /speedy.home /apps /oracle**

You can mount any directory from an exported directory hierarchy. In this example, speedy exports /export and the local system mounts /export/progs and /export/oracle. The following commands manually mount the directory hierarchies one time:

mount speedy:/home /speedy.home

mount -o ro,nosuid speedy:/export/progs /apps

mount -o ro speedy:/export/oracle /oracle

The first command mounts the /home directory hierarchy from speedy on the local directory /speedy.home. The second and third commands use the -o ro option to force a readonly mount. The second command adds the nosuid option, which forces setuid executables in the mounted directory hierarchy to run with regular permissions on the local system. If you receive the error mount: RPC: Program not registered, it may mean NFS is not running on the server. By default, directory hierarchies are mounted read-write, assuming the NFS server is exporting them with read-write permissions.

- **nosuid option:**

If a user could run a setuid program, that user has the power of Superuser. This ability should be limited. Unless you know that a user will need to run a program with setuid permissions from a mounted directory hierarchy, always mount a directory hierarchy with the nosuid option. For example, you would need to mount a directory hierarchy with setuid privileges when a diskless workstation has its root partition mounted using NFS.

- **nodev option**

Mounting a device file creates another potential security hole. Although the best policy is not to mount untrustworthy directory hierarchies, it is not always possible to implement this policy. Unless a user needs to use a device on a mounted directory hierarchy, mount directory hierarchies with the nodev option, which prevents character and block special files on the mounted directory hierarchy from being used as devices.

- **fstab file**

If you mount directory hierarchies frequently, you can add entries for the directory hierarchies to the **/etc/fstab** file.

```
$ cat /etc/fstab
```

```
...
```

```
speedy:/home          /speedy.home    nfs - 0 0
speedy:/export/progs   /apps           nfsr,nosuid 0 0
speedy:/export/oracle  /oracle         nfs r 0 0
```

A file that is mounted using NFS is always type nfs on the local system, regardless of what type it is on the remote system. Typically, you do not run fsck on or back up an NFS directory hierarchy. The entries in the third, fifth, and sixth columns of fstab are usually nfs (filesystem type), 0 (do not back up this directory hierarchy with dump), and 0 (do not run fsck on this directory hierarchy). The options for mounting an NFS directory hierarchy differ from those for mounting an ext3 or other type of filesystem.

- **umount: Unmounts a Remote Directory Hierarchy**

Use umount to unmount a remote directory hierarchy the same way you would unmount a local filesystem

mount: Mounts a Directory Hierarchy

The mount utility associates a directory hierarchy with a mount point (a directory). You can use mount to mount an NFS (remote) directory hierarchy.

Attribute Caching

File attributes, which are stored in a file's inode, provide information about a file, such as file modification time, size, links, and owner. File attributes do not include the data stored in a file. Typically file attributes do not change very often for an ordinary file; they change even less often for a directory file. Even the size attribute does not change with every write instruction: When a client is writing to an NFS-mounted file, several write instructions may be given before the data is transferred to the server

ac (noac) (attribute cache): It permits attribute caching (default). The noac option disables attribute caching. Although noac slows the server, it avoids stale attributes when two NFS clients actively write to a common directory hierarchy.

acdirmax=n (attribute cache directory file maximum): The n is the number of seconds, at a maximum, that NFS waits before refreshing directory file attributes (default is 60 seconds).

acdirmin=n (attribute cache directory file minimum): The n is the number of seconds, at a minimum, that NFS waits before refreshing directory file attributes (default is 30 seconds).

acregmax=n (attribute cache regular file maximum): The n is the number of seconds, at a maximum, that NFS waits before refreshing regular file attributes (default is 60 seconds).

acregmin=n (attribute cache regular file minimum): The n is the number of seconds, at a minimum, that NFS waits before refreshing regular file attributes (default is 3 seconds).

actimeo=n (attribute cache timeout): This option sets acregmin, acregmax, acdirmin, and acdirmax to n seconds (without this option, each individual option takes on its assigned or default value).

Error Handling

Various options control what NFS does when the server does not respond or when an I/O error occurs. To allow for a mount point located on a mounted device, a missing mount point is treated as a timeout.

- **fg (bg) (foreground):** The option fg retries failed NFS mount attempts in the foreground (default). The bg (background) option retries failed NFS mount attempts in the background.
- **hard (soft) :** This option displays server not responding on the console on a major timeout and keeps retrying (default). The soft option reports an I/O error to the calling program on a major timeout. In general, it is not advisable to use soft. As the mount man page says of soft, "Usually it just causes lots of trouble."
- **nointr (intr) (no interrupt):** This option does not allow a signal to interrupt a file operation on a hard mounted directory hierarchy when a major timeout occurs (default). The intr option allows this type of interrupt.
- **retrans=n (retransmission value):** After n minor timeouts, NFS generates a major timeout (default is 3). A major timeout aborts the operation or displays server not responding on the console, depending on whether hard or soft is set.
- **retry=n (retry value):** The number of minutes that NFS retries a mount operation before giving up (default is 10,000).
- **timeo=n (timeout value):** The n is the number of tenths of a second that NFS waits before retransmitting following an RPC, or minor, timeout (default is 7). The value is increased at each timeout to a maximum of 60 seconds or until a major timeout occurs. On a busy network, in case of a slow server, or when the request passes through multiple routers/gateways, increasing this value may improve performance.

Miscellaneous Options

- **lock (nolock):** Permits NFS locking (default). The nolock option disables NFS locking (does not start the lockd daemon) and is useful with older servers that do not support NFS locking.
- **mounthost=name:** The name of the host running mountd, the NFS mount daemon.
- **mountport=n:** The port used by mountd.
- **nodev (no device):** Causes mounted device files not to function as devices.
- **port=n:** The port used to connect to the NFS server (defaults to 2049 if the NFS daemon is not registered with rpcbind/portmap). When n=0 (default), NFS queries rpcbind/portmap on the server to determine the port.
- **rsize=n (read block size)** The number of bytes read at one time from an NFS server. The default block size is 4096.
- **wsize=n (write block size):** The number of bytes written at one time to an NFS server. The default block size is 4096.

- **tcp:** Use TCP in place of the default UDP protocol for an NFS mount. This option may improve performance on a congested network; however, some NFS servers support UDP only.
- **udp:** Use the default UDP protocol for an NFS mount.

Improving Performance:

- **hard/soft:** Several parameters can affect the performance of NFS, especially over slow connections such as a line with a lot of traffic or one controlled by a modem. If you have a slow connection, make sure **hard** is set (this is the default) so that timeouts do not abort program execution.
- **Block size:** One of the easiest ways to improve NFS performance is to increase the block size—that is, the number of bytes NFS transfers at a time. The default of 4096 is low for a fast connection using modern hardware. Try increasing **rsize** and **wsiz** to 8192 or higher. Experiment until you find the optimal block size. Unmount and mount the directory hierarchy each time you change an option.
- **Timeouts:** NFS waits the amount of time specified by the **timeo** option for a response to a transmission. If it does not receive a response in this amount of time, it sends another transmission. The second transmission uses bandwidth that, over a slow connection, may slow things down further. You may be able to increase performance by increasing **timeo**.

/etc/fstab: Mounts Directory Hierarchies Automatically

The **/etc/fstab** file lists directory hierarchies that the system mounts automatically as it comes up. This example line from **fstab** mounts **grape's /gc1** filesystem on the **/grape.gc1** mount point:

```
grape:/gc1      /grape.gc1      nfs,rsize=8192,wsiz=8192 0      0
```

A mount point should be an empty, local directory. Files in a mount point are hidden when a directory hierarchy is mounted on it. The type of a filesystem mounted using NFS is always **nfs**, regardless of its type on the local system. You can increase the **rsize** and **wsiz** options to improve performance. This example from **fstab** mounts a filesystem from **speedy**:

```
speedy:/export  /speedy.export  nfs,timeo=50,hard      0      0
```

Because the local system connects to **speedy** over a slow connection, **timeo** is increased to 5 seconds (50 tenths of a second). This example from **fstab** shows a remote-mounted home directory. Because **speedy** is a local server and is connected via a reliable, high-speed connection, **timeo** is decreased and **rsize** and **wsiz** are increased substantially:

```
speedy:/export/home /home nfs,timeo=4,rsize=16384,wsiz=16384 0      0
```

14.2 Setting Up an NFS Server

The prerequisites for setting up NFS server is to install the following packages:

nfs-utils

system-config-nfs (optional)

After this run **chkconfig** to cause **nfs** to start when the system enters multiuser mode:

```
# /sbin/chkconfig nfs on
```

The **nfs** can be started by:

```
# /sbin/service nfs start
```

The `nfsinit` script starts `mountd`, `nfsd`, and `rquotad`. Under RHEL, the `portmap` daemon must be running to enable reliable file locking.

JumpStart II: Configuring an NFS Server Using `system-config-nfs`:

To display the NFS Server Configuration window, enter the command `system-config-nfs` or select Main Menu: System | Administration | Server Settings | NFS. From this window you can generate an `/etc/exports` file, which is almost all there is to setting up an NFS server. The `system-config-nfs` utility allows you to specify which directory hierarchies are shared and how they are shared using NFS. Each exported hierarchy is called a share. To add a share, click Add on the toolbar. To modify a share, highlight the share and click Properties on the toolbar. Clicking Add displays the Add NFS Share window, while clicking Properties displays the Edit NFS Share window. The Add/Edit NFS Share window has three tabs: Basic, General Options, and User Access. These are given as:

- **Basic tab:** You can specify the pathname of the root of the shared directory hierarchy, the names or IP addresses of the systems (hosts) that the hierarchy will be shared with, and whether users from the specified systems will be able to write to the shared files. The selections in the other two tabs correspond to options that you can specify in the `/etc/exports` file.

- **General Options tab:** In this tab, make the following changes:

Allow connections from ports 1024 and higher: **insecure**

Allow insecure file locking: **no_auth_nlm or insecure_locks**

Disable subtree checking: **no_subtree_check**

Sync write operations on request: **sync**

Force sync of write operations immediately: **no_wdelay**

Hide filesystems beneath: **nohide**

Export only if mounted: **mountpoint**

- **User Access tab:** In this tab, make the following changes:

Treat remote root user as local root: **no_root_squash**

Treat all client users as anonymous users: **all_squash**

Local user ID for anonymous users: **anonuid**

Local group ID for anonymous users: **anongid**

After making the changes you want, click OK to close the Add/Edit NFS Share window and click OK again to close the NFS Server Configuration window. There is no need to restart any daemons.

Exporting a Directory Hierarchy

Exporting a directory hierarchy makes the directory hierarchy available for mounting by a client on the network. “Exported” does not mean “mounted”: When a directory hierarchy is exported, it is placed in the list of directory hierarchies that can be mounted by other systems. An exported directory hierarchy may be mounted (or not) at any given time. A server holds three lists of exported directory hierarchies:

- **/etc/exports**— Access control list for exported directory hierarchies. The system administrator can modify this file by editing it or by running `system-config-nfs`.

- **/var/lib/nfs/xtab** — Access control list for exported directory hierarchies. Initialized from /etc/exports when the system is brought up. Read by mountd when a client asks to mount a directory hierarchy. Modified by exportfs as directory hierarchies are mounted and unmounted by NFS.
- **Kernel's export table** — List of active exported directory hierarchies. The kernel obtains this information from /var/lib/nfs/xtab. You can display this table by giving the command `cat /proc/fs/nfs/exports`. The /etc/exports file is the access control list for exported directory hierarchies that NFS clients can mount; it is the only file you need to edit to set up an NFS server. The exports file controls the following aspects:
 - 1) Which clients can access files on the server?
 - 2) Which directory hierarchies on the server each client can access
 - 3) How each client can access each directory hierarchy
 - 4) How client usernames are mapped to server usernames
 - 5) Various NFS parameters

Each line in the exports file has the following format:

export-point client1(options) [client2(options) ...]

where export-point is the absolute pathname of the root directory of the directory hierarchy to be exported, client1-n is the name of one or more clients or is one or more IP addresses, separated by SPACES, that are allowed to mount the export-point. You can either use system-config-nfs to make changes to exports or you can edit this file directly. The following simple exports file gives grape read and write access and gives speedy readonly access to the files in /home:

cat /etc/exports

/home grape(rw,no_subtree_check)

/home speedy(ro,no_subtree_check)

General options

- **auth_nlm (no_auth_nlm) or secure_locks (insecure_locks)**: This causes the server to require authentication of lock requests (using the NLM [NFS Lock Manager] protocol). Use no_auth_nlm for older clients when you find that only files that anyone can read can be locked.
- **mountpoint[=path]**: It allows a directory to be exported only if it has been mounted. This option prevents a mount point that does not have a directory hierarchy mounted on it from being exported and prevents the underlying mount point from being exported.
- **nohide (hide)**: When a server exports two directory hierarchies, one of which is mounted on the other, a client must mount both directory hierarchies explicitly to access both. When the second (child) directory hierarchy is not explicitly mounted, its mount point appears as an empty directory and the directory hierarchy is hidden. The nohide option causes the underlying second directory hierarchy to appear when it is not explicitly mounted, but this option does not work in all cases.
- **ro (rw) (readonly)**: It permits only read requests on an NFS directory hierarchy. Use rw to permit read and write requests.
- **secure (insecure)**: It requires that NFS requests originate on a privileged port so that a program without root permissions cannot mount a directory hierarchy. This option does not guarantee a secure connection.

- **subtree_check (no_subtree_check):** It checks subtrees for valid files. Assume that you have an exported directory hierarchy that has its root below the root of the filesystem that holds it (that is, an exported subdirectory of a filesystem). When the NFS server receives a request for a file in that directory hierarchy, it performs a subtree check to confirm the file is in the exported directory hierarchy.
- **sync (async) (synchronize):** It specifies that the server is to reply to requests only after disk changes made by the request are written to disk. The **async option** specifies that the server does not have to wait for information to be written to disk and can improve performance, albeit at the cost of possible data corruption if the server crashes or the connection is interrupted.
- **Wdelay (no_wdelay) (write delay):** Causes the server to delay committing write requests when it anticipates that another, related request follows, thereby improving performance by committing multiple write requests within a single operation. The **no_wdelay option** does not delay committing write requests and can improve performance when the server receives multiple, small, unrelated requests.

User ID Mapping Options

Each user has a UID number and a primary GID number on the local system. The local `/etc/passwd` and `/etc/group` files map these numbers to names. When a user makes a request of an NFS server, the server uses these numbers to identify the user on the remote system. It raises several issues: the user may not have the same ID numbers on both systems and may therefore have owner access to files of another user. You may not want the root user on the client system to have owner access to root-owned files on the server. You may not want a remote user to have owner access to some important system files that are not owned by root (such as those owned by `bin`).

Owner access means that the remote user can execute, remove, or – worse – modify the file. NFS gives you two ways to deal with these cases:

- You can use the `root_squash` option to map the ID number of the root user on a client to the `nfsnobody` user on the server.
- You can use the `all_squash` option to map all NFS users on the client to `nfsnobody` on the server

NIS and NFS: When you use NIS for user authorization, users automatically have the same UIDs on both systems. If you are using NFS on a large network, it is a good idea to use a directory service such as LDAP or NIS for authorization. Without such a service, you must synchronize the `passwd` files on all the systems manually.

root_squash (no_root_squash): It maps requests from root on a remote system so that they appear to come from the UID for `nfsnobody`, an unprivileged user on the local system, or as specified by a `nonuid`. It does not affect other sensitive UIDs such as `bin`. The `no_root_squash` option turns off this mapping so that requests from root appear to come from root.

no_all_squash (all_squash): It does not change the mapping of users making requests of the NFS server. The `all_squash` option maps requests from all users, not just root, on remote systems to appear to come from the UID for `nfsnobody`, an unprivileged user on the local system, or as specified by a `nonuid`. This option is useful for controlling access to exported public FTP, news, and other directories.

anonuid=un and anongid=gn: It sets the UID or the GID of the anonymous account to un or gn, respectively. NFS uses these accounts when it does not recognize an incoming UID or GID and when instructed to do so by root_squash or all_squash.

showmount:

It displays NFS status information. Without any options, the showmount utility displays a list of systems that are allowed to mount local directories. To display information for a remote system, give the name of the remote system as an argument. You typically use showmount to display a list of directory hierarchies that a server is exporting. The information that showmount provides may not be complete, however, because it depends on mountd and trusts that remote servers are reporting accurately. In the following example, bravo and grape can mount local directories, but you do not know which ones:

```
# /usr/sbin/showmount
```

Hosts on localhost:

```
bravo.tcorp.com
```

```
grape.tcorp.com
```

If showmount displays an error such as RPC: Program not registered, NFS is not running on the server. Start NFS on the server with the nfsinit script

-a (all): It tells which directories are mounted by which remote systems. This information is stored in /etc/exports.

```
# /usr/sbin/showmount -a
```

All mount points on localhost:

```
bravo.tcorp.com:/home
```

```
grape.tcorp.com:/home
```

-e (exports): It displays a list of exported directories.

```
# /usr/sbin/showmount -e
```

Export list for localhost:

```
/home bravo.tcorp.com,grape.tcorp.com
```

exportfs: Maintains the List of Exported Directory Hierarchies

The exportfs utility maintains the kernel's list of exported directory hierarchies. Without changing /etc/exports, exportfs can add to or remove from the list of exported directory hierarchies. An exportfs command has the following format:

```
/usr/sbin/exportfs [options] [client:dir ...]
```

where options is one or more options (as detailed in the next section), client is the name of the system that dir is exported to, and dir is the absolute pathname of the directory at the root of the directory hierarchy being exported. The system executes the following command when it comes up (it is in the nfsinit script). This command reexports the entries in /etc/exports and removes invalid entries from /var/lib/nfs/xtab so that /var/lib/nfs/xtab is synchronized with /etc/exports:

```
# exportfs -r
```

Options: There are various options which we can use:

- **-a (all):** Exports directory hierarchies specified in /etc/exports. This option does not unexport entries you have removed from exports (that is, it does not remove invalid entries from /var/lib/nfs/xtab); use -r to perform this task.

- **-i (ignore):** Ignores /etc/exports; uses what is specified on the command line only.
- **-o (options):** Specifies options. You can specify options following -o the same way you do in the exports file. For example, `exportfs -i -o ro speedy:/home/sam exports /home/sam` on the local system to speedy for readonly access.
- **-r (reexport):** Reexports the entries in /etc/exports and removes invalid entries from /var/lib/nfs/xtab so that /var/lib/nfs/xtab is synchronized with /etc/exports.
- **-u (unexport)** Makes an exported directory hierarchy no longer exported. If a directory hierarchy is mounted when you unexport it, you will see the message Stale NFS file handle if you try to access the directory hierarchy from the remote system.
- **-v (verbose):** Provides more information. Displays export options when you use `exportfs` to display export information.

14.3 Testing the Server Setup

From the server, run the `nfsinit` script with an argument of status. If all is well, the system displays something like the following:

```
# /sbin/service nfs status
```

```
rpc.mountd (pid 15795) is running...
```

```
nfsd (pid 15813 15812 15811 15810 15809 15808 15807 15806) is running...
```

```
rpc.rquotad (pid 15784) is running...
```

Next, from the server, use `rpcinfo` to make sure NFS is registered with `rpcbind/portmap`:

```
$ /usr/sbin/rpcinfo -p localhost | grep nfs
```

```
100003 2 udp 2049 nfs
```

```
100003 3 udp 2049 nfs
```

Repeat the preceding command from the client, replacing `localhost` with the name of the server. The results should be the same. Finally, try mounting directory hierarchies from remote systems and verify access.

14.4 Automount: Automatically Mounts Directory Hierarchies

With distributed computing, when you log in on any system on the network, all of your files, including startup scripts, are available. In a distributed computing environment, all systems are commonly able to mount all directory hierarchies on all servers: Whichever system you log in on, your home directory is waiting for you. As an example, assume that `/home/alex` is a remote directory hierarchy that is mounted on demand. When you issue the command `ls /home/alex`, `autofs` goes to work: It looks in the `/etc/auto.home` map, finds that `alex` is a key that says to mount `bravo:/export/home/alex`, and mounts the remote directory hierarchy. Once the directory hierarchy is mounted, `ls` displays the list of files you want to see. If you give the command `ls /home` after this mounting sequence, `ls` shows that `alex` is present within the `/home` directory. The `df` utility shows that `alex` is mounted from `bravo`.

Prerequisites

The prerequisites is to install the following package: **autofs**

Run `chkconfig` to cause `autofs` to start when the system enters multiuser mode:

```
# /sbin/chkconfig autofs on
```

Start `autofs`:

```
# /sbin/service autofs start
```

An **autofs directory hierarchy** is like any other directory hierarchy, but remains unmounted until it is needed, at which time the system mounts it automatically (demand mounting). The system unmounts an autofs directory hierarchy when it is no longer needed – by default after five minutes of inactivity. Automatically mounted directory hierarchies are an important part of administering a large collection of systems in a consistent way. The automount daemon is particularly useful when an installation includes many servers or many directory hierarchies. It also helps to remove server-server dependencies. When you boot a system that uses traditional fstab-based mounts and an NFS server is down, the system can take a long time to come up as it waits for the server to time out. Similarly, when you have two servers, each mounting directory hierarchies from the other, and both systems are down, both may hang as they are brought up and each tries to mount a directory hierarchy from the other. This situation is called a server-server dependency. The automount facility gets around these issues by mounting a directory hierarchy from another system only when a process tries to access it. When a process attempts to access one of the directories within an unmounted autofs directory hierarchy, the kernel notifies the automount daemon, which mounts the directory hierarchy. You must give a command, such as `cd /home/alex`, that accesses the autofs mount point (in this case `/home/alex`) to create the demand that causes automount to mount the autofs directory hierarchy so you can see it. Before you issue the `cd` command, alex does not appear to be in `/home`. The main file that controls the behavior of automount is `/etc/auto.master`. Example:

```
# cat /etc/auto.master
```

```
/free1  /etc/auto.misc  --timeout      60
/free2  /etc/auto.misc2 --timeout      60
```

The `auto.master` file has three columns.

- The first column names the parent of the autofs mount point—the location where the autofs directory hierarchy is to be mounted (`/free1` and `/free2` in the example are not mount points but will hold the mount points when the directory hierarchies are mounted).
- The second column names the files, called map files, that store supplemental configuration information.
- The optional third column holds mount options for map entries that do not specify an option.

Although the map files can have any names, one is traditionally named `auto.misc`. Following are the two map files specified in `auto.master`:

```
# cat /etc/auto.misc
```

```
sam -fstype=ext3 :/dev/sda8
```

```
# cat /etc/auto.misc2
```

```
helen -fstype=ext3 :/dev/sda9
```

Before the new setup can work, you must create directories for the parents of the mount points (`/free1` and `/free2` in the preceding example) and start (or restart) the automount daemon using the `autofsinit` script. The following command displays information about configured and active autofs mount points:

```
# /sbin/service autofs status
```

Summary:

- NFS runs on UNIX, DOS, Windows, VMS, Linux, and more. Files on the remote computer (the fileserver) appear as if they are present on the local system (the client). The physical location of a file is irrelevant to an NFS user.

- NFS is based on the trusted-host paradigm and therefore has all the security shortcomings that plague other services based on this paradigm.
- To display the NFS Server Configuration window, enter the command `system-config-nfs` or select Main Menu: System | Administration | Server Settings | NFS.
- A file that is mounted using NFS is always type `nfs` on the local system, regardless of what type it is on the remote system.
- The `mount` utility associates a directory hierarchy with a mount point (a directory). You can use `mount` to mount an NFS (remote) directory hierarchy.
- A server holds three lists of exported directory hierarchies. These are: `/etc/exports`, `/var/lib/nfs/xtab` and kernel's export table.

Keywords:

- **NFS:** NFS stands for Network Filesystem protocol. It is a UNIX de facto standard originally developed by Sun Microsystems. It allows a server to share selected local directory hierarchies with client systems on a heterogeneous network.
- **NFS Server:** The NFS server translates commands from the client into operations on the server's filesystem.
- **df utility:** The `df` utility displays a list of the directory hierarchies available on the system, along with the amount of disk space, free and used, on each.
- **portmap utility:** The `portmap` utility must be running to enable reliable file locking.
- **umount:** It unmounts a remote directory hierarchy. Use `umount` to unmount a remote directory hierarchy the same way you would unmount a local filesystem.
- **Share:** The `system-config-nfs` utility allows you to specify which directory hierarchies are shared and how they are shared using NFS. Each exported hierarchy is called a share.
- **exportfs:** The `exportfs` utility maintains the kernel's list of exported directory hierarchies. Without changing `/etc/exports`, `exportfs` can add to or remove from the list of exported directory hierarchies.

Self Assessment

1. The problem in NFS security is
 - A. NFS is encrypted
 - B. NFS is not encrypted
 - C. NFS does not respond
 - D. None of the above
2. Which of these options disables a signal to interrupt a file operation on hardmounted directory hierarchy?
 - A. `intr`
 - B. `nointr`
 - C. `unintr`
 - D. None of the above

3. Which of these options displays a list of exported directories?
 - A. -a
 - B. -e
 - C. -list
 - D. None of the above

4. In FEDORA, which utility is used for setting up an NFS client?
 - A. portmap
 - B. rpcbind
 - C. setupnfs
 - D. None of the above

5. By default block size in wsize and rsize is _____
 - A. 128
 - B. 256
 - C. 1468
 - D. 4096

6. Which of these options are used to mount a remote directory hierarchy?
 - A. mount
 - B. automount
 - C. Both of the above
 - D. None of the above

7. Which utility displays a list of directory hierarchies available on the system?
 - A. df
 - B. dir
 - C. dirhier
 - D. None of the above
8. Which of these options disables the attribute caching?
 - A. ac
 - B. noac
 - C. unac
 - D. None of the above

9. Which of these options permits only read access on an NFS directory hierarchy?
 - A. r
 - B. ro
 - C. rw
 - D. or

10. NFS
 - A. Reduces the storage requirement

- B. Boosts efficiency
 - C. Reduces administration workload
 - D. All of the above
11. If a user can run a `setuid` program, that user
- A. Is a normal user
 - B. Has the power of a superuser
 - C. Hides the identity
 - D. None of the above
12. Kernel's export table consists of
- A. Active exported directory hierarchies
 - B. Inactive exported directory hierarchies
 - C. Both active and inactive
 - D. None of the above
13. NFS stands for
- A. Network FileSystem
 - B. Not a FileSystem
 - C. New Filesystem
 - D. None of the above
14. In RHEL, which utility is used for setting up an NFS client?
- A. `portmap`
 - B. `rpcbind`
 - C. `setupnfs`
 - D. None of the above
15. The NFS performance can be improved by _____
- A. Increasing the block size
 - B. Decreasing the block size
 - C. Block size should remain constant
 - D. None of the above

Answers for Self Assessment

- | | | | | |
|-------|-------|-------|-------|-------|
| 1. B | 2. B | 3. B | 4. B | 5. D |
| 6. C | 7. A | 8. B | 9. B | 10. D |
| 11. B | 12. C | 13. A | 14. A | 15. A |

Review Questions:

1. What is NFS? Explain the flow of data from client to server with the help of a diagram.
2. Explain the various features of NFS.
3. How to set up an NFS client? Explain.
4. Explain the utility which is used to mount a directory hierarchy with various options available with it.
5. What is error handling? Also explain various options available with it.
6. How to set up an NFS server?
7. How to export a directory hierarchy? Explain the lists associated with this.
8. How to test the server setup? Explain autofs.

**Further Readings**

Mark G Sobell, A Practical Guide to Fedora and RedHat Enterprise Linux, Fifth Edition, Prentice Hall

**Web Links**

<https://cloud.netapp.com/blog/azure-anf-blg-linux-nfs-server-how-to-set-up-server-and-client>

LOVELY PROFESSIONAL UNIVERSITY

Jalandhar-Delhi G.T. Road (NH-1)

Phagwara, Punjab (India)-144411

For Enquiry: +91-1824-300360

Fax.: +91-1824-506111

Email: odl@lpu.co.in